

T24 Core Banking Guide

Banking Domain + T24 Flow + Safe Backend Integration

Tài liệu học chuyên sâu cho backend developer: hiểu đủ nghiệp vụ ngân hàng, hiểu luồng Temenos Transact/T24, và biết thiết kế tích hợp an toàn với core banking.

Banking Domain

T24 Flow

Safe Integration

Use Cases

Mục tiêu của tài liệu

Sau khi đọc, bạn có thể nhìn một sơ đồ ngân hàng và hiểu: request từ IB/MB/Card đi qua đâu, T24 xử lý module nào, posting/accounting tạo ra như thế nào, timeout phải xử lý ra sao, khi nào cần reversal, khi nào cần reconciliation, và backend cần những guardrail nào để không gây double posting.

Phiên bản: 1.0 - PDF only

Ngữ cảnh: Core Banking, Payment Gateway, ESB, Temenos Transact/T24

Đối tượng: Java backend developer mới vào banking domain

Mục lục

1. Bức tranh tổng quan: Core Banking T24 là gì?	5
1.1. T24 không phải chỉ là database tài khoản	5
1.2. Đặt T24 vào kiến trúc ngân hàng	5
2. Banking domain foundation cho người mới	6
2.1. Customer/CIF là gốc của quan hệ ngân hàng	6
2.2. Account không chỉ là số tài khoản	6
2.3. Product, arrangement và contract	6
2.4. Deposit, loan, card, payment nhìn từ góc độ ngân hàng	7
3. Accounting trong core banking: hiểu đủ để không bị mù domain	7
3.1. Debit/Credit theo góc nhìn ngân hàng	7
3.2. Các ví dụ posting nền tảng	8
3.3. Journal, GL và balance không được lệch	8
4. T24/Temenos Transact: các khái niệm cần biết	8
4.1. Module map	8
4.2. Những từ khóa T24 hay gặp trong dự án	9
4.3. Arrangement Architecture - AA	9
5. Kiến trúc tích hợp an toàn với T24	10
5.1. Các lớp trong integration	10
5.2. API Gateway vs ESB vs Core Adapter	10
5.3. Integration methods thường gặp	10
5.4. Message formats: JSON, OFS, ISO 8583, ISO 20022	11
6. Luồng nghiệp vụ T24 thường gặp	11
6.1. Balance inquiry	11
6.2. Internal transfer cùng ngân hàng	11

6.3. Interbank/payment gateway flow	12
6.4. Term deposit lifecycle	12
6.5. Loan lifecycle	13
7. Safe backend integration patterns	13
7.1. Idempotency	14
Pseudo-code idempotency cho transfer	14
7.2. Timeout, retry và query status	14
7.3. Transaction state machine	14
7.4. Outbox pattern cho sự kiện sau posting	15
7.5. Audit log và data masking	15
8. COB/EOD, reconciliation và vận hành production	15
8.1. COB/EOD là gì?	15
8.2. Reconciliation	16
9. Use case thực tế hay gặp trong dự án Core/T24	16
10. Checklist thiết kế backend an toàn khi gọi Core/T24	17
10.1. Checklist API tài chính online	17
10.2. Checklist database/schema cho core adapter	17
10.3. Error mapping strategy	17
11. Lộ trình học để “đủ làm việc” với T24/Core	18
11.1. Lộ trình 30 ngày	18
11.2. Câu hỏi phỏng vấn/thực chiến	18
12. Glossary thuật ngữ Core Banking/T24	19
13. Appendix - Mermaid source cho các sơ đồ	20
Kiến trúc tổng quan	20
Chuỗi xử lý chuyển tiền nội bộ	20
Trạng thái giao dịch an toàn	20

Vòng đời tiền gửi có kỳ hạn	21
COB/EOD	21
14. Nguồn tham khảo	21

1. Bức tranh tổng quan: Core Banking T24 là gì?

Temenos T24 là tên gọi quen thuộc trong nhiều dự án ngân hàng; tên sản phẩm core banking hiện nay của Temenos là Temenos Transact. Temenos công bố việc đổi tên từ T24 Transact sang Temenos Transact để phản ánh các thay đổi về khả năng xử lý giao dịch, mở rộng và hướng công nghệ mới [S1].

Trong tài liệu này, "T24" được dùng theo cách mà các team dự án thường gọi: hệ thống core banking chịu trách nhiệm giữ sổ cái nghiệp vụ, sổ dư tài khoản, sản phẩm ngân hàng, bút toán kế toán và các xử lý cuối ngày.

Câu nhớ nhanh

Digital banking là nơi khách hàng thao tác. T24/core banking là nơi tiền thật sự được kiểm tra, ghi nhận và hạch toán. Backend integration là lớp đảm bảo việc gọi vào core không bị trùng, không sai trạng thái, không mất dấu vết kiểm toán.

1.1. T24 không phải chỉ là database tài khoản

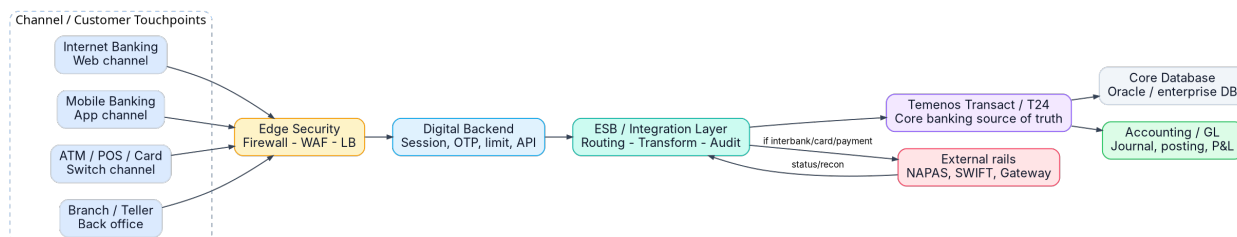
Một hệ thống web thông thường thường xoay quanh CRUD: tạo, đọc, sửa, xóa dữ liệu. Core banking thì khác: mỗi giao dịch tài chính cần kiểm tra điều kiện nghiệp vụ, cập nhật sổ dư, tạo posting debit/credit, phát sinh journal, phản ánh vào general ledger, sau đó còn đi qua batch cuối ngày và đối soát.

Góc nhìn	Web system thông thường	Core banking/T24
Dữ liệu chính	User, order, product, cart	Customer/CIF, account, arrangement, balance, journal, GL
Lỗi nguy hiểm nhất	Sai dữ liệu hiển thị hoặc sai trạng thái đơn hàng	Double posting, mất tiền, lệch ledger, sai báo cáo tài chính
Retry	Có thể retry tương đối đơn giản	Retry phải có idempotency + query status
Audit	Log phục vụ debug	Audit là yêu cầu vận hành, kiểm toán và pháp lý
Batch	Có thể có hoặc không	COB/EOD là một phần quan trọng của vòng đời hệ thống

1.2. Đặt T24 vào kiến trúc ngân hàng

Trong kiến trúc ngân hàng, người dùng không gọi trực tiếp T24. Request thường bắt đầu từ Mobile Banking, Internet Banking, ATM/POS/Card, quầy giao dịch hoặc đối tác. Sau đó request đi qua firewall/WAF, API Gateway hoặc backend kênh số, rồi qua ESB hoặc integration layer để chuyển đổi format và route vào module core phù hợp.

Sơ đồ 1 - Kiến trúc tổng quan Core Banking + T24



Sơ đồ Mermaid-style: Channel -> EdgeLayer -> Digital Backend -> ESB -> T24 -> DB/GL/External rails.

Temenos mô tả Core Banking là nền tảng modular, có khả năng triển khai linh hoạt và phục vụ nhiều phân khúc như retail, business, corporate, treasury và nhiều capability ngân hàng khác [S2]. Temenos Developer Portal

cũng cung cấp nhóm API cho Core Banking như arrangements, accounts, deposits, lending, payments, financial accounting, limits và data [S3].

2. Banking domain foundation cho người mới

2.1. Customer/CIF là gốc của quan hệ ngân hàng

CIF là Customer Information File - hồ sơ khách hàng trong core banking. Một CIF có thể có nhiều tài khoản, nhiều sản phẩm, nhiều khoản vay, nhiều hợp đồng tiền gửi, nhiều thẻ, nhiều quan hệ đồng sở hữu.

Trong backend, bạn thường thấy customerId ở digital system khác với CIF ở core. Vì vậy cần map rõ: user đăng nhập app không nhất thiết chính là CIF duy nhất; một user có thể đại diện cho cá nhân, doanh nghiệp, hoặc nhiều tài khoản được ủy quyền.

Khái niệm	Giải thích	Điểm backend cần chú ý
CIF	Hồ sơ khách hàng trong core	Không dùng phone/email làm khóa nghiệp vụ chính khi gọi core
KYC	Xác minh khách hàng	Trạng thái KYC ảnh hưởng quyền mở tài khoản/giao dịch
Segment	Nhóm khách hàng: retail, SME, corporate, VIP	Phí, lãi, limit có thể khác theo segment
Relationship	Quan hệ đồng sở hữu, người đại diện, doanh nghiệp	Kiểm tra entitlement trước khi cho user thao tác

2.2. Account không chỉ là số tài khoản

Account trong core có số tài khoản, loại sản phẩm, currency, branch/company, trạng thái, balance, hold amount, hạn mức thấu chi, cấu hình statement, cấu hình lãi/phí. Một account có thể active, dormant, frozen, closed, blocked debit, blocked credit hoặc chỉ được giao dịch theo điều kiện.

Loại balance	Ý nghĩa	Ví dụ
Ledger balance	Số dư sổ sách sau các posting đã ghi nhận	10.500.000 VND
Available balance	Số dư có thể sử dụng sau khi trừ hold/block/uncleared	10.000.000 VND
Hold amount	Khoản giữ chỗ, chưa được dùng	500.000 VND giữ cho card authorization
Overdraft limit	Hạn mức âm được phép	Được chi vượt 20 triệu nếu có hợp đồng thấu chi

Sai lầm hay gặp

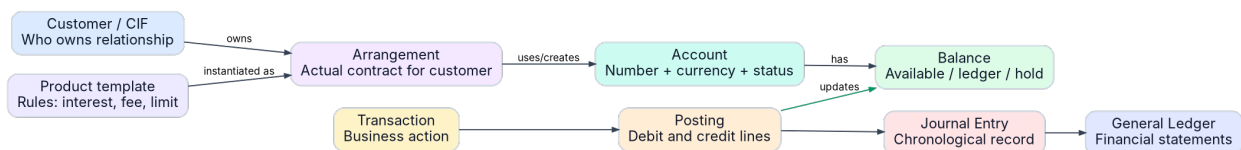
Không được chỉ kiểm tra ledger balance rồi cho chuyển tiền. Backend nên dùng available balance do core trả về hoặc để core tự validate trong giao dịch posting. Available balance mới phản ánh các khoản hold, block, limit và rule nghiệp vụ.

2.3. Product, arrangement và contract

Trong banking, product là mẫu sản phẩm: tài khoản thanh toán thường, tài khoản VIP, tiền gửi 6 tháng, vay mua nhà, vay tín chấp. Arrangement là hợp đồng/sản phẩm cụ thể của một khách hàng được tạo từ product đó.

Temenos mô tả Arrangement Architecture là framework của Transact để tạo và quản lý các sản phẩm như loans, deposits, accounts; framework này dựa trên business component có thể tái sử dụng để cấu hình nhiều sản phẩm khác nhau [S3].

Sơ đồ 2 - Mô hình khái niệm Customer -> Product -> Arrangement -> Posting -> GL



Đây là xương sống tư duy khi đọc T24: ai là khách hàng, đang dùng sản phẩm nào, hợp đồng thực tế là gì, giao dịch tạo posting nào.

2.4. Deposit, loan, card, payment nhìn từ góc độ ngân hàng

Module	Ngân hàng nhìn như thế nào?	Ví dụ nghiệp vụ
Deposit/CASA	Tiền khách gửi là liability của ngân hàng	Mở tài khoản, nạp tiền, rút tiền, tính lãi tiền gửi
Loan/Lending	Khoản cho vay là asset của ngân hàng	Giải ngân, thu nợ, tính lãi, xử lý quá hạn
Card	Kênh chi tiêu/thanh toán, có authorization/settlement	POS, ATM, refund, reversal, chargeback
Payment	Dịch chuyển giá trị qua account hoặc external rail	Internal transfer, interbank, bill payment, merchant payment
Treasury/FX	Quản lý tiền tệ, thanh khoản, tỷ giá	Mua bán ngoại tệ, revaluation, nostro/vostro

3. Accounting trong core banking: hiểu đủ để không bị mù domain

Trong core banking, hầu hết giao dịch tiền đều cần tạo bút toán kế toán. Note Accounting bạn cung cấp đã ghi đúng các ý nền tảng: journal là bản ghi giao dịch theo thứ tự thời gian; general ledger chứa các tài khoản kế toán; balance sheet luôn cân bằng debit và credit; Profit & Loss ghi nhận profit/loss và thường reset theo năm tài chính [U2].

3.1. Debit/Credit theo góc nhìn ngân hàng

Loại tài khoản kế toán	Tăng ở bên nào?	Ví dụ trong ngân hàng
Asset	Debit	Cash, nostro, khoản vay khách hàng
Liability	Credit	Tiền gửi khách hàng, khoản phải trả
Equity	Credit	Vốn chủ sở hữu
Income	Credit	Thu lãi vay, thu phí chuyển tiền
Expense	Debit	Chi lãi tiền gửi, chi phí vận hành

Điểm đảo ngược rất quan trọng

Với khách hàng, số dư tiền gửi là tài sản của khách hàng. Nhưng với ngân hàng, tiền gửi khách hàng là nghĩa vụ phải trả, tức liability. Vì vậy khi khách nộp tiền, ngân hàng ghi tăng asset đồng thời tăng liability.

3.2. Các ví dụ posting nền tảng

Use case	Debit	Credit	Ý nghĩa
Khách nộp 10 triệu	Cash/Nostro Asset +10m	Customer Deposit Liability +10m	Bank có thêm tiền, đồng thời nợ khách nhiều hơn
Chuyển A -> B cùng bank	Deposit Liability - Account A	Deposit Liability - Account B	Giảm nghĩa vụ với A, tăng nghĩa vụ với B
Giải ngân vay 100 triệu	Loan Asset +100m	Customer Deposit Liability +100m	Bank có tài sản là khoản phải thu, khách có tiền trong account
Tính lãi tiền gửi	Interest Expense	Interest Payable/Deposit Liability	Chi phí lãi của ngân hàng tăng
Thu lãi vay	Customer Deposit Liability	Interest Income	Bank thu tiền khách và ghi nhận doanh thu

Temenos Developer mô tả accounting events được phát ra cùng với business events cho nhiều banking product. Ví dụ khi execute credit transaction trên savings account thì business event về credit transaction và accounting event cập nhật journal entries được phát ra cùng API response [S4].

3.3. Journal, GL và balance không được lệch

Một hệ thống core tốt không chỉ cập nhật số dư account. Nó phải giữ được liên kết giữa transaction, posting lines, journal entries và GL. Nếu transaction thành công nhưng journal không có, hoặc journal có nhưng balance không đối, hoặc GL tổng hợp không cân, đó là vấn đề production nghiêm trọng.

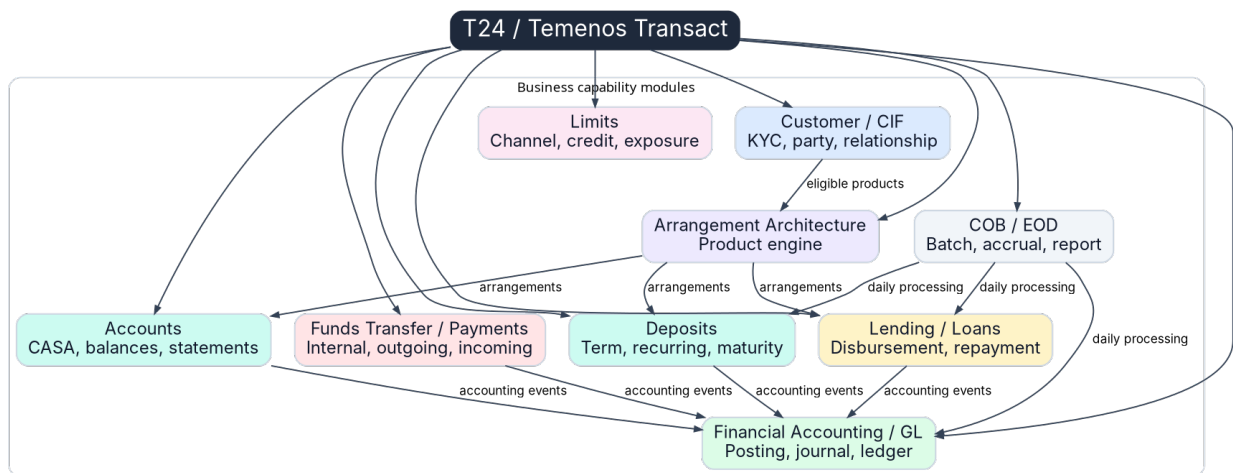
Layer dữ liệu	Lưu cái gì?	Dùng để làm gì?
Transaction	Thông tin nghiệp vụ: ai, làm gì, bao nhiêu, trạng thái	Hiện thị lịch sử giao dịch, tra cứu dispute
Posting line	Dòng debit/credit chi tiết	Cập nhật balance và kiểm tra double-entry
Journal entry	Nhật ký kế toán theo thời gian	Audit, accounting trace
General Ledger	Tổng hợp theo tài khoản kế toán	Financial report, balance sheet, P&L

4. T24/Temenos Transact: các khái niệm cần biết

4.1. Module map

Ở mức backend integration, bạn không cần trở thành T24 consultant ngay. Nhưng bạn cần hiểu các module thường gặp để đọc log, đọc mapping, trao đổi với core team và thiết kế API an toàn.

Sơ đồ 3 - Module map trong T24/Temenos Transact



Các module không độc lập tuyệt đối; Customer, AA, Account, Deposit, Lending, Funds Transfer và GL liên kết chặt qua product, arrangement và accounting events.

4.2. Những từ khóa T24 hay gặp trong dự án

Từ khóa	Ý nghĩa thực dụng cho backend developer
CUSTOMER	Hồ sơ khách hàng/CIF trong T24.
ACCOUNT	Tài khoản ngân hàng, gồm account number, status, currency, category/product, balance.
FUNDS.TRANSFER	Ứng dụng/khái niệm liên quan chuyển tiền trong T24 truyền thống.
AA.PRODUCT	Cấu hình sản phẩm trong Arrangement Architecture.
AA.ARRANGEMENT	Hợp đồng/sản phẩm thực tế của khách hàng được tạo từ AA product.
ACTIVITY	Một hành động trên arrangement: open, fund, amend, withdraw, close, rollover.
VERSION	Màn hình/biến thể nhập liệu T24 cho một application.
ENQUIRY	Cơ chế truy vấn/tra cứu dữ liệu trong T24.
OFS	Open Financial Service - dạng message/interface để tích hợp vào T24 trong nhiều triển khai.
COB/EOD	Close of Business/End of Day batch processing.
LOCAL.REF	Trường mở rộng/custom theo triển khai ngân hàng.
EB.ERROR	Nhóm lỗi/thông báo trong T24, thường cần mapping sang lỗi business để hiểu.

4.3. Arrangement Architecture - AA

AA có thể xem như product engine. Thay vì hard-code từng sản phẩm, ngân hàng cấu hình product từ các property/condition: interest, charge, settlement, schedule, statement, limit, accounting. Khi khách hàng mua/mở sản phẩm, T24 tạo arrangement cụ thể.

Khái niệm AA	Cách hiểu đơn giản	Ví dụ
Product line	Dòng sản phẩm lớn	Accounts, deposits, lending
Product	Mẫu sản phẩm	TD_6M_VND, CASA_STANDARD
Property	Nhóm đặc tính	Interest, charge, settlement
Condition	Điều kiện cụ thể trong property	Lãi 5.5%/năm, miễn phí nếu VIP
Arrangement	Hợp đồng thực tế	Khách A gửi 100 triệu kỳ hạn 6 tháng
Activity	Hành động lên arrangement	fund, withdraw, rollover, close

Cách trao đổi với core team

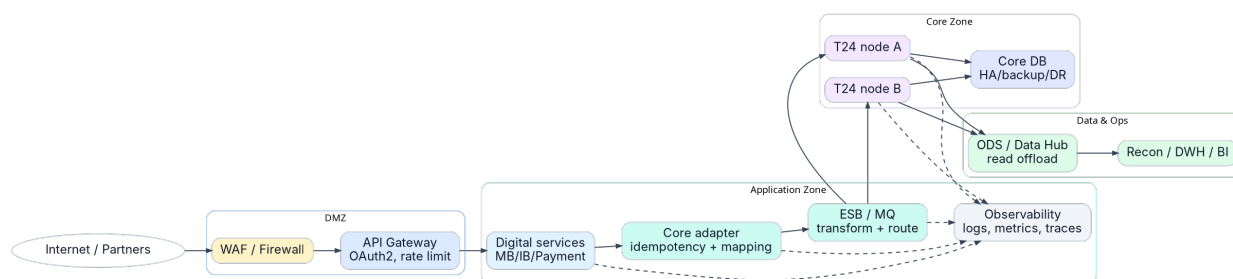
Khi tích hợp một API như “mở tiết kiệm”, đừng chỉ hỏi endpoint. Hãy hỏi product code là gì, arrangement activity nào được gọi, settlement account lấy ở đâu, phí/lãi/early withdrawal rule ở product hay backend, và response trả arrangementId/accountId nào.

5. Kiến trúc tích hợp an toàn với T24

5.1. Các lớp trong integration

Một thiết kế an toàn thường tách rõ: channel backend xử lý trải nghiệm người dùng, core adapter xử lý contract với core, ESB/middleware xử lý routing/transform, T24 xử lý source of truth, còn ODS/Data Hub phục vụ query/reporting nếu phù hợp.

Sơ đồ 4 - Deployment architecture hiện đại



Không nên để app public gọi thẳng vào core zone. Core zone cần được bảo vệ bằng network segmentation, adapter, ESB/MQ và audit.

5.2. API Gateway vs ESB vs Core Adapter

Thành phần	Vai trò chính	Không nên làm gì?
API Gateway	AuthN/AuthZ gateway, rate limit, route API, WAF integration	Không chứa business posting logic phức tạp
Digital Backend	Session, OTP, user context, UX workflow, validation nhẹ	Không tự quyết định số dư cuối cùng nếu core phải là source of truth
Core Adapter	Canonical API, idempotency, mapping T24, timeout handling	Không bỏ qua query status khi timeout
ESB/Middleware	Route, transform, protocol conversion, audit, sometimes orchestration	Không biến thành nơi chứa tất cả business logic không kiểm soát
T24/Core	Validate nghiệp vụ lõi, post transaction, update balance/GL	Không bị query/report nặng trực tiếp bởi nhiều hệ thống ngoài

5.3. Integration methods thường gặp

Kiểu tích hợp	Dùng khi nào?	Backend cần chú ý
REST/API	Triển khai mới, API-first, sandbox/documentation rõ	Versioning, auth, error mapping, semantic retries
OFS/message	T24 truyền thống hoặc integration legacy	Format, delimiter, field order, response parser, simulator test
Fixed-length/TCP	Core/ESB legacy, hiệu năng cao, protocol riêng	Padding, encoding, timeout, correlation ID
MQ/event	Asynchronous flow, notification, data sync	At-least-once delivery, idempotent consumer, outbox
File/batch	EOD, settlement, reconciliation	File checksum, sequence, rerun policy, audit trail

Temenos Data Hub có Operational Data Store gần real-time để phục vụ API/use case vận hành và offload reporting/enquiry workloads khỏi core; điều này phản ánh một pattern phổ biến: không để mọi query nặng

đánh trực tiếp vào core transaction engine [S8].

5.4. Message formats: JSON, OFS, ISO 8583, ISO 20022

IB/MB hiện đại thường nói JSON/REST. Card/ATM/POS có thể liên quan ISO 8583 - chuẩn định nghĩa interface và format cho message giao dịch thẻ giữa acquirer và issuer [S9]. Payment/liên ngân hàng quốc tế hoặc modern payment rails có thể dùng ISO 20022, một chuẩn/methodology chung cho thông điệp tài chính [S10].

Format	Hay gặp ở đâu?	Vấn đề backend hay gặp
JSON/REST	Mobile/Internet banking, partner API	Schema versioning, validation, idempotency header
OFS/T24 message	T24 legacy integration	Parser, escape characters, field mapping, response code
Fixed-length	Core legacy, host-to-host	Padding, length, encoding, null/blank semantics
ISO 8583	ATM/POS/Card switch	MTI, bitmap, data element, reversal/advice
ISO 20022 XML	Payment clearing/SWIFT/new rails	Rich data, schema validation, mapping to internal model

6. Luồng nghiệp vụ T24 thường gặp

6.1. Balance inquiry

Balance inquiry là read-only nhưng vẫn cần kiểm soát entitlement. User đang đăng nhập app có quyền xem account này không? Account có thuộc CIF của user không? Đây là nơi dễ bị lỗi Broken Object Level Authorization nếu backend chỉ nhận accountId từ client rồi query thẳng. OWASP API Security Top 10 2023 xếp Broken Object Level Authorization là rủi ro API số 1 [S12].

Sơ đồ 5 - Balance inquiry sequence

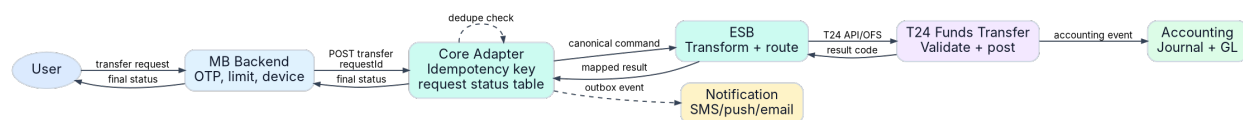


Bước	Xử lý	Guardrail
1	App gọi GET balance	Không tin accountId từ client
2	Backend lấy user context/CIF mapping	Check ownership/entitlement
3	Adapter gọi T24 hoặc ODS	TraceId + timeout budget
4	Trả available/ledger/hold balance	Mask dữ liệu nhạy cảm, không expose internal code

6.2. Internal transfer cùng ngân hàng

Internal transfer là flow quan trọng nhất để học integration core. Backend phải tạo requestId/idempotencyKey, kiểm tra OTP/limit ở kênh, gửi lệnh vào core, sau đó chỉ trả thành công khi trạng thái đã rõ. Nếu timeout, tuyệt đối không retry mù.

Sơ đồ 6 - Internal transfer sequence

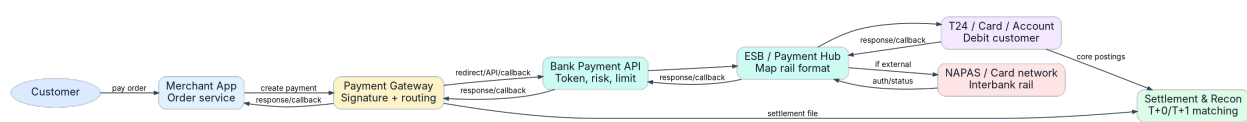


Pha	Core/T24 xử lý	Backend cần làm an toàn
Validate	Account active, đủ available balance, currency, limit, product rule	Validate nhẹ trước core nhưng không thay core quyết định
Post	Debit source, credit destination, fee/tax nếu có	Một requestId không được tạo nhiều posting
Commit	Transaction + journal + balance được ghi nhận	Chỉ coi thành công khi core response hoặc query status xác nhận
Notify	SMS/push/email	Dùng outbox, không để lỗi notification rollback posting core

6.3. Interbank/payment gateway flow

Interbank và merchant payment phức tạp hơn internal transfer vì có thêm external rail: NAPAS/card network/payment gateway/SWIFT. Trạng thái có thể pending, accepted, rejected, settled, reversed hoặc cần đối soát sau cut-off.

Sơ đồ 7 - Payment gateway/interbank flow



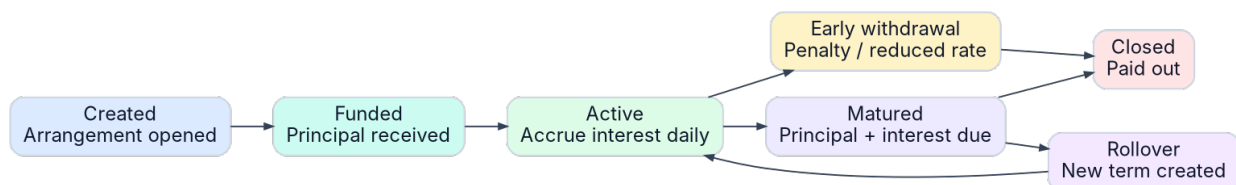
Payment gateway không chỉ gọi API. Nó cần signature, callback verification, settlement, reconciliation và dispute handling.

Use case	Điểm khó	Pattern nên có
Merchant payment	Callback giả, callback đến trước response, duplicate callback	Verify signature + idempotent callback handler
Interbank transfer	Core đã debit nhưng rail timeout	Query status/advice/reversal theo rule rail
Refund	Refund không phải xóa giao dịch cũ	Tạo transaction mới liên kết originalRef
Settlement	Gateway báo success nhưng tiền settlement lệch	Đối soát file + break management

6.4. Term deposit lifecycle

Temenos có lifecycle guide cho term deposit gồm các sự kiện điển hình như term deposit account creation, funding, early withdrawal và manual rollover; lãi tích lũy/capitalization có thể là hoạt động hệ thống tùy tài liệu/lifecycle [S6].

Sơ đồ 8 - Vòng đời tiền gửi có kỳ hạn

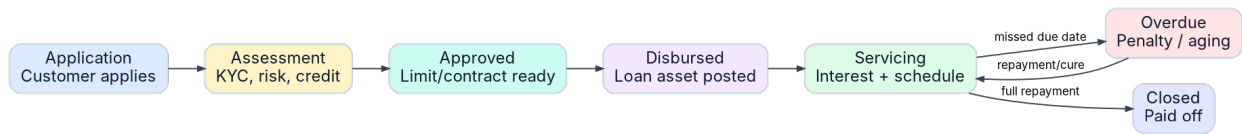


Term deposit không dừng ở "mở sổ". Nó có funding, accrual, maturity, early withdrawal, rollover và close.

Event	Nghịệp vụ	Posting/accounting thường gặp
Creation	Tạo arrangement/account tiền gửi	Có thể chưa phát sinh tiền nếu chưa fund
Funding	Trích tiền từ settlement account sang term deposit	Debit customer CASA, credit term deposit liability
Accrual	Tính lãi phát sinh theo ngày	Debit interest expense, credit interest payable/accrual
Maturity	Đến hạn trả gốc/lãi hoặc rollover	Debit term deposit liability, credit settlement account
Early withdrawal	Rút trước hạn	Tính lại lãi/phí phạt, reverse/accrual adjustment nếu cần

6.5. Loan lifecycle

Sơ đồ 9 - Loan lifecycle



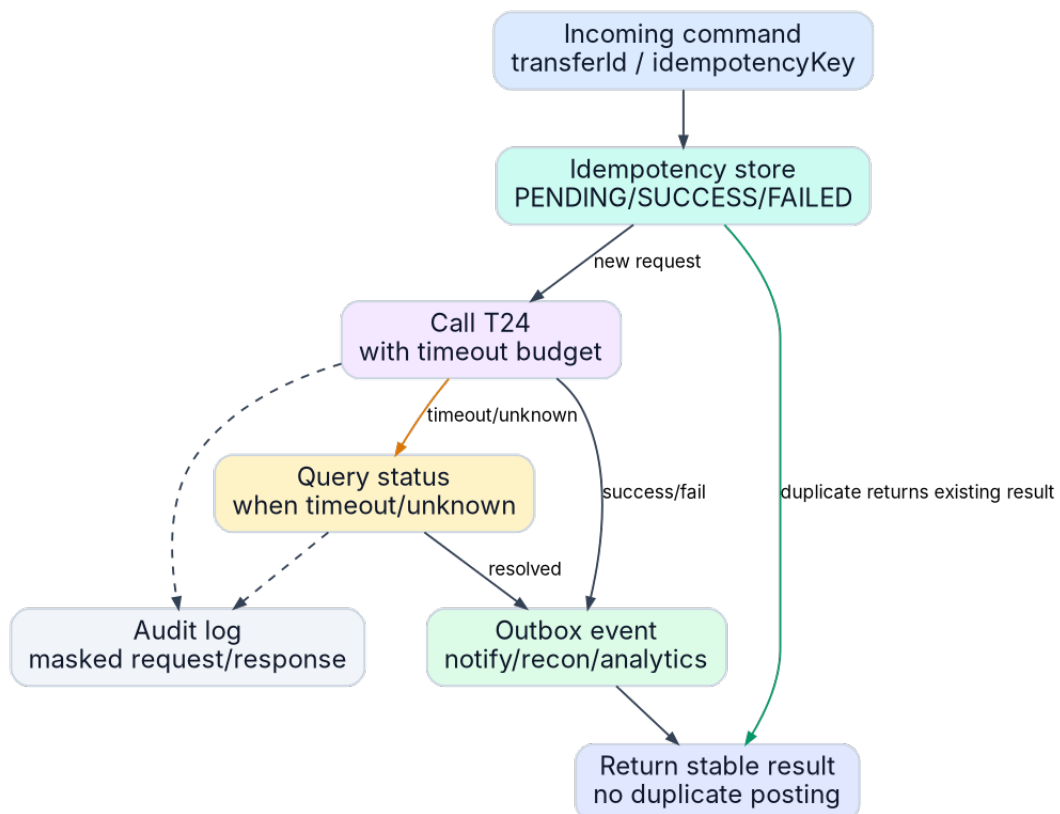
Loan nhìn từ ngân hàng là asset; deposit nhìn từ ngân hàng là liability. Đây là nền tảng để hiểu posting.

Pha	Nhiệm vụ	Điểm tích hợp/backend
Application	Khách yêu cầu vay	Digital có thể lưu hồ sơ tạm, chưa tạo loan asset
Approval	Credit/risk phê duyệt	Maker-checker, approval limit, document status
Disbursement	Giải ngân	Tạo loan asset và credit tiền vào account khách
Servicing	Tính lãi, lịch trả nợ	Query schedule, due amount, payoff amount
Repayment	Khách trả nợ	Allocation: fee/penalty/interest/principal theo rule
Overdue	Quá hạn	COB aging, penalty, provisioning, collection workflow

7. Safe backend integration patterns

Phần này là trọng tâm với backend developer. Nhiệm vụ của bạn không chỉ là gọi API T24. Bạn phải thiết kế sao cho lỗi mạng, timeout, retry, duplicate message, callback trùng, user bấm nhiều lần, service restart, queue redelivery đều không làm core bị double posting hoặc mất dấu vết.

Sơ đồ 10 - Safe integration pattern



Một request tài chính nên đi qua idempotency store, audit log, timeout strategy, query status và outbox.

7.1. Idempotency

Idempotency nghĩa là cùng một ý định nghiệp vụ được gửi nhiều lần thì kết quả tài chính chỉ được ghi nhận một lần. Trong banking, idempotencyKey/requestId không phải chi tiết phụ; nó là guardrail chống double posting.

Trường	Vai trò	Gợi ý
requestId	Định danh request từ channel	Sinh một lần trước khi gọi core
businessRef	Mã nghiệp vụ hiển thị/tra cứu	Có thể dùng làm transaction reference
coreRef	Mã giao dịch do T24 trả	Lưu sau khi core nhận/post
idempotency status	PENDING/SUCCESS/FAILED/UNKNOWN	Dùng để trả kết quả ổn định cho duplicate request
request hash	Hash payload chính	Ngăn cùng requestId nhưng payload khác

Pseudo-code idempotency cho transfer

```
function transfer(command):
  existing = repo.findByRequestId(command.requestId)
  if existing exists:
    if existing.requestHash != hash(command): reject("same id, different payload")
    return existing.stableResult

  repo.insert(requestId, hash, status="PENDING")
  try:
    coreResult = t24.postTransfer(command)
    repo.markSuccess(requestId, coreRef=coreResult.coreRef)
    outbox.publish("TRANSFER_POSTED", requestId)
    return success(coreResult)
  catch Timeout:
    repo.markUnknown(requestId)
    status = t24.queryStatus(command.requestId or businessRef)
    return resolveAndPersist(status)
  catch BusinessError as e:
    repo.markFailed(requestId, e.code)
    return failed(e)
```

7.2. Timeout, retry và query status

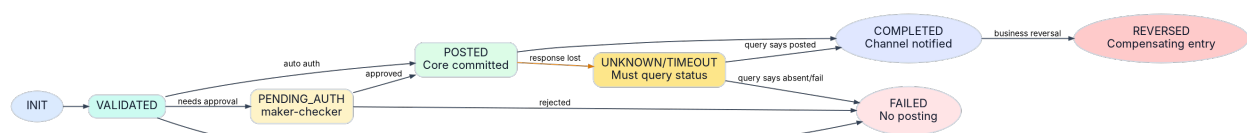
Tình huống	Không được làm	Nên làm
Timeout trước khi core nhận	Retry vô hạn	Retry có giới hạn với cùng idempotencyKey
Timeout sau khi core đã post	Tạo request mới	Query status bằng requestRef/coreRef trước
Response code không rõ	Map đại thành failed	Đưa vào UNKNOWN/PENDING_RECON và reconcile
ESB trả lỗi kỹ thuật	Assume core fail	Kiểm tra core/ESB log hoặc query status

Nguyên tắc vàng

Với giao dịch tài chính, unknown không đồng nghĩa failed. Unknown nghĩa là backend chưa biết core đã post hay chưa. Hành động đúng là query status hoặc đưa vào pending reconciliation, không phải tạo giao dịch mới.

7.3. Transaction state machine

Sơ đồ 11 - Transaction state machine



State machine giúp tránh logic lẫn lộn giữa failed, timeout, posted và reversed.

Trong thiết kế database backend, nên có bảng transaction_command hoặc payment_order lưu trạng thái riêng của kênh. Trạng thái này không nhất thiết giống trạng thái core nhưng phải map được: local request đang PENDING/UNKNOWN, coreRef nào, có cần query status không, có cần reconcile không.

7.4. Outbox pattern cho sự kiện sau posting

Sau khi core post thành công, nhiều việc phụ phát sinh: gửi notification, cập nhật loyalty, gửi Kafka event, đẩy DWH, tạo receipt. Những việc này không được làm cho core posting bị rollback hoặc bị gọi lại. Outbox pattern lưu event cùng transaction local, rồi worker gửi event sau.

Không dùng outbox	Dùng outbox
Core post success, gửi SMS fail -> user tưởng fail hoặc retry nguy hiểm	Core post success, lưu event pending, worker retry SMS độc lập
Service crash sau core success trước khi publish Kafka -> mất event	Event nằm trong DB outbox và được publish sau khi service recover
Duplicate message downstream gây side effect	Consumer cũng idempotent theo eventId

7.5. Audit log và data masking

Banking log phải đủ để trace nhưng không được lộ dữ liệu nhạy cảm. PCI DSS định nghĩa yêu cầu bảo vệ môi trường lưu trữ/xử lý/truyền dữ liệu tài khoản thanh toán, và có yêu cầu bảo vệ dữ liệu chủ thẻ [S11]. Với API nói chung, OWASP cũng nhấn mạnh rủi ro về authentication, authorization, excessive data exposure, logging/monitoring [S12].

Nên log	Không nên log rõ
traceId, requestId, businessRef, coreRef	PIN, OTP, password, CVV
masked account: 001***789	full card PAN nếu không có yêu cầu và kiểm soát PCI
error code, latency, endpoint, service name	token, session cookie, private key
payload sau masking ở mức audit vault	raw request chứa PII không kiểm soát

8. COB/EOD, reconciliation và vận hành production

8.1. COB/EOD là gì?

COB/EOD là quy trình đóng ngày nghiệp vụ. Đây không đơn giản là cron job. Nó có thể bao gồm cut-off, kiểm tra dữ liệu, tính lãi, thu phí, loan aging, xử lý đáo hạn, tổng hợp GL, xuất report, đồng bộ DWH và mở ngày mới.

Sơ đồ 12 - COB/EOD flow



COB/EOD là nơi nhiều nghiệp vụ hệ thống tự phát sinh mà không đến từ user click trực tiếp.

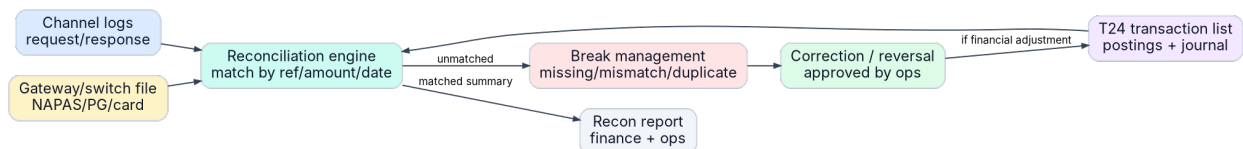
Job	Mục đích	Vấn đề production hay gặp
Interest accrual	Tính lãi phát sinh hằng ngày	Sai rate/product config dẫn đến sai lãi hàng loạt
Fee charging	Thu phí định kỳ	Thu trùng phí nếu rerun không idempotent
Loan aging	Cập nhật quá hạn	Không chuyển overdue đúng ngày, sai báo cáo nợ
Maturity processing	Xử lý term deposit đến hạn	Rollover sai instruction hoặc trả lãi sai settlement account
GL consolidation	Tổng hợp bút toán	GL imbalance, suspense account tăng bất thường

Job	Mục đích	Vấn đề production hay gặp
Reports/DWH	Xuất dữ liệu vận hành/quản trị	Report lệch core nếu sync thiếu

8.2. Reconciliation

Reconciliation là đối soát giữa các hệ thống: channel, core, gateway/switch, bank statement, settlement file. Mục tiêu là phát hiện giao dịch thiếu, trùng, lệch amount, lệch status hoặc đã success ở một bên nhưng pending/fail ở bên khác.

Sơ đồ 13 - Reconciliation flow



Đối soát không phải report đẹp. Nó là cơ chế phát hiện và xử lý financial breaks.

Loại break	Ví dụ	Hướng xử lý
Missing in core	Gateway success nhưng core không có posting	Điều tra, post adjustment hoặc refund tùy rule
Missing in gateway	Core debit nhưng gateway không nhận	Query rail, reversal hoặc manual repair
Amount mismatch	Core 1.003.300, gateway 1.000.000	Kiểm tra fee/tax/settlement mapping
Duplicate	Hai posting cùng requestId/businessRef	Reverse một giao dịch, RCA idempotency
Status mismatch	Core posted, channel failed	Cập nhật channel status, notify customer nếu cần

9. Use case thực tế hay gặp trong dự án Core/T24

Bảng dưới đây là playbook thực dụng cho backend developer. Khi nhận task mới, hãy xác định use case thuộc nhóm nào, core object nào liên quan, dữ liệu nào là source of truth, và lỗi nào nguy hiểm nhất.

Use case	Flow chính	T24/core liên quan	Rủi ro	Checklist backend
Login + list accounts	User login -> map user to CIF -> list accounts	Customer, Account, Arrangement/holding	BOLA: xem account người khác	Entitlement, account ownership, mask account
Query balance	Channel -> adapter -> T24/ODS	Account balance	Balance cũ/sai loại balance	available vs ledger, traceId, no sensitive log
Internal transfer	Debit A -> credit B -> journal	Funds Transfer, Account, GL	Double debit, timeout unknown	idempotency, query status, stable state
Interbank transfer	Debit source -> send rail -> pending/settled	Payment/FT, settlement account	Rail timeout, recon break	status machine, reversal, recon file
Merchant payment	Order -> gateway -> bank -> callback	Payment API, account/card	Fake callback, duplicate callback	signature, idempotent callback, settlement
Open CASA account	Check CIF -> choose product -> create arrangement/account	AA product, account arrangement	Wrong product/config	validate product, return accountId/arrangementId
Open term deposit	Create TD -> fund from settlement account	AA deposit, term deposit	Fund fail after create	stateful flow, rollback/close if needed
Early withdrawal TD	Request withdraw -> calculate penalty -> post	Deposit activity, interest/fee	Wrong interest reversal	quote before execute, maker-checker if high amount
Loan disbursement	Approve -> create loan -> credit customer	Loan/Lending, account, GL	Disburse twice	approval state, idempotency, coreRef
Loan repayment	Debit customer -> allocate fee/interest/principal	Loan servicing, schedule	Wrong allocation order	quote payoff/due amount, clear status after post

Use case	Flow chính	T24/core liên quan	Rủi ro	Checklist backend
Fee charging	Online fee or batch fee	Charge condition, GL income	Duplicate fee on rerun	idempotent fee event, audit fee rule
Reversal	Create compensating transaction linked to original	FT/payment/accounting	Deleting old transaction	never delete, link originalRef, approval
EOD report	COB -> export data -> BI/regulatory	GL, transaction, ODS/DWH	Report lệch business date	snapshot date, recon counts, rerun policy

10. Checklist thiết kế backend an toàn khi gọi Core/T24

10.1. Checklist API tài chính online

Nhóm	Câu hỏi kiểm tra
Identity & entitlement	User có quyền thao tác account/CIF này không? Có kiểm tra ở server không?
Idempotency	Có requestId duy nhất không? Duplicate request trả kết quả cũ hay tạo posting mới?
Timeout	Nếu core timeout thì trạng thái local là gì? Có query status không?
Retry	Retry có cùng idempotency key không? Có retry với lỗi business không?
Amount/currency	Có validate decimal, currency, minor unit, max/min amount, FX rate không?
Core source of truth	Balance/fee/interest do core quyết định hay backend tự tính? Nếu tự tính thì có reconciliation không?
Audit	Log có traceId/requestId/coreRef không? Có masking PII/card/token không?
Error mapping	Lỗi core có được map sang lỗi nghiệp vụ ổn định không? Có lộ internal code không?
Reversal	Có flow đảo giao dịch không? Ai được phép? Có maker-checker không?
Recon	Giao dịch có xuất hiện trong báo cáo đối soát không? Có status pending_recon không?

10.2. Checklist database/schema cho core adapter

Table/field	Mục đích
transaction_request	Lưu requestId, hash, user, amount, status, timestamps
core_call_log	Lưu endpoint/message type, request/response masked, latency, error
transaction_reference	Map localRef, businessRef, coreRef, externalRef
outbox_event	Event hậu xử lý: notification, Kafka, recon, email
recon_item	Bản ghi chờ/đã đối soát với external/core
error_mapping	Map core error code -> business code/message/action

10.3. Error mapping strategy

Một lỗi T24/ESB không nên được trả nguyên xi cho mobile app. Nên có mapping table phân loại lỗi: lỗi validation có thể trả ngay, lỗi technical có thể retry/query status, lỗi unknown cần pending, lỗi security phải reject và audit.

Loại lỗi	Ví dụ	Action
Business validation	Insufficient funds, account closed, limit exceeded	FAILED, trả message nghiệp vụ
Authentication/authorization	User không có quyền	REJECTED, security audit
Technical retryable	Connection reset trước khi gửi request	Retry có kiểm soát cùng requestId

Loại lỗi	Ví dụ	Action
Technical unknown	Timeout sau khi gửi request	UNKNOWN, query status
Core accepted pending	Need approval, waiting settlement	PENDING/PENDING_AUTH
Duplicate	RequestId đã success	Return existing stable result

11. Lộ trình học để “đủ làm việc” với T24/Core

11.1. Lộ trình 30 ngày

Giai đoạn	Mục tiêu	Bạn cần tự trả lời được
Ngày 1-5: Foundation	CIF, account, balance, debit/credit, GL	Vì sao deposit là liability, loan là asset?
Ngày 6-10: Deposit & account	CASA, term deposit, product, arrangement	Mở account/TD cần những bước nào?
Ngày 11-15: Funds transfer	Internal/interbank transfer, fee, status	Timeout sau khi core posted xử lý sao?
Ngày 16-20: Integration	API/OFS/ESB, idempotency, error mapping	Adapter cần lưu những reference nào?
Ngày 21-25: COB/recon	EOD, accrual, charge, settlement, breaks	Recon phát hiện những loại lệch nào?
Ngày 26-30: Production mindset	Security, monitoring, audit, incident	Làm sao chứng minh một giao dịch đã/không được post?

11.2. Câu hỏi phỏng vấn/thực chiến

Core banking khác web CRUD ở điểm nào? - Trả lời bằng posting, GL, audit, reconciliation, EOD.

Tại sao timeout không được coi là failed? - Vì core có thể đã post nhưng response bị mất.

IdempotencyKey lưu ở đâu và dùng thế nào? - Lưu cùng request hash/status/coreRef, duplicate trả stable result.

Available balance khác ledger balance thế nào? - Available trừ hold/block/uncleared, ledger là sổ sách đã post.

Reversal khác delete transaction thế nào? - Reversal là giao dịch bù trừ có audit, không xóa giao dịch gốc.

AA trong T24 là gì? - Product engine dựa trên reusable business components để tạo accounts/deposits/loans arrangements.

COB/EOD xử lý gì? - Accrual, charge, maturity, loan aging, GL consolidation, report, data sync.

12. Glossary thuật ngữ Core Banking/T24

Thuật ngữ	Giải thích
AA	Arrangement Architecture - framework/product engine của Temenos để quản lý product và arrangement.
Account	Tài khoản ngân hàng dùng để ghi nhận balance và giao dịch.
Accounting event	Sự kiện kế toán phát sinh khi giao dịch nghiệp vụ tạo ảnh hưởng tài chính.
Accrual	Ghi nhận lãi/chi phí/doanh thu phát sinh theo thời gian dù chưa thanh toán.
Available balance	Số dư có thể sử dụng sau khi trừ hold/block/uncleared amount.
BOD	Beginning of Day - mở ngày nghiệp vụ mới.
BOLA	Broken Object Level Authorization - lỗi API cho phép truy cập object không thuộc quyền user.
CASA	Current Account/Savings Account - tài khoản thanh toán/tiết kiệm không kỳ hạn.
Charge	Phí giao dịch hoặc phí định kỳ.
CIF	Customer Information File - hồ sơ khách hàng trong core.
COB	Close of Business - đóng ngày nghiệp vụ, chạy batch cuối ngày.
Core adapter	Service trung gian chuẩn hóa API, idempotency, mapping và timeout khi gọi core.
CoreRef	Mã tham chiếu giao dịch do core/T24 trả.
Credit	Bên Có trong kế toán; liability/equity/income tăng bên credit.
Debit	Bên Nợ trong kế toán; asset/expense tăng bên debit.
Deposit	Tiền gửi/tài khoản tiền gửi; với ngân hàng là liability.
Double posting	Một ý định giao dịch bị ghi nhận tài chính nhiều lần.
EOD	End of Day - xử lý cuối ngày.
ESB	Enterprise Service Bus - lớp điều phối, chuyển đổi message/protocol và audit integration.
Fee income	Doanh thu phí của ngân hàng.
Funds Transfer	Chuyển tiền giữa tài khoản hoặc qua payment rail.
GL	General Ledger - sổ cái kế toán tổng hợp.
Hold amount	Khoản bị giữ tạm, làm giảm available balance.
Idempotency	Cơ chế đảm bảo retry/duplicate không tạo nhiều kết quả tài chính.
Interbank	Giao dịch giữa các ngân hàng qua rail/switch/clearing.
Journal	Nhật ký kế toán theo thứ tự thời gian.
Ledger balance	Số dư sổ sách sau posting đã ghi nhận.
Liability	Nợ phải trả của ngân hàng, ví dụ tiền gửi khách hàng.
Loan asset	Khoản cho vay là tài sản của ngân hàng.
Maker-checker	Cơ chế một người tạo, người khác kiểm tra/duyet.
Maturity	Ngày đáo hạn của term deposit/loan/contract.
NAPAS	Hạ tầng/switch thanh toán nội địa tại Việt Nam, thường gặp trong thẻ/chuyển tiền.
Nostro	Tài khoản ngân hàng mở tại ngân hàng khác, thường phục vụ thanh toán/FX.
ODS	Operational Data Store - kho dữ liệu vận hành gần real-time để offload query/report.
OFS	Open Financial Service - kiểu message/interface truyền thống để tích hợp T24 ở nhiều triển khai.
Outbox pattern	Lưu event trong DB rồi worker publish, tránh mất event khi service crash.
Payment gateway	Cổng thanh toán kết nối merchant, bank, wallet/card/rail và callback/settlement.
Posting	Các dòng debit/credit làm thay đổi balance và kế toán.
Product	Mẫu sản phẩm ngân hàng có rule lãi/phí/limit/accounting.

Thuật ngữ	Giải thích
Reconciliation	Đối soát dữ liệu giữa core, channel, gateway/switch và settlement file.
RequestId	Mã định danh request từ hệ thống gọi, dùng cho trace và idempotency.
Reversal	Giao dịch đảo/bù trừ có audit, không phải xóa giao dịch cũ.
Settlement	Quá trình quyết toán tiền giữa các bên sau giao dịch.
Suspense account	Tài khoản tạm treo khi chưa xác định đúng nơi hạch toán.
Term deposit	Tiền gửi có kỳ hạn, có maturity và rule rút trước hạn.
TraceId	Mã theo dõi xuyên suốt request qua nhiều service.
Unknown status	Trạng thái chưa rõ do timeout/lỗi kỹ thuật; cần query status/recon.

13. Appendix - Mermaid source cho các sơ đồ

Các sơ đồ trong tài liệu được trình bày trực quan trong PDF. Phần dưới đây là Mermaid source tương ứng để bạn copy vào Mermaid Live Editor hoặc tài liệu Markdown nội bộ.

Kiến trúc tổng quan

```

flowchart LR
  CH[IB / MB / ATM / POS / Branch] --> EDGE[Firewall / WAF / API Gateway]
  EDGE --> DIG[Digital Backend]
  DIG --> ESB[ESB / Integration Layer]
  ESB --> T24[T24 / Temenos Transact]
  T24 --> DB[(Core DB)]
  T24 --> GL[Accounting / GL]
  ESB --> EXT[NAPAS / SWIFT / Payment Gateway]
```

Chuỗi xử lý chuyển tiền nội bộ

```

sequenceDiagram
    participant U as User
    participant MB as MB Backend
    participant AD as Core Adapter
    participant ESB as ESB
    participant T24 as T24 Funds Transfer
    participant GL as Accounting/GL
    U->>MB: Submit transfer + OTP
    MB->>AD: POST /transfers with requestId
    AD->>AD: Check idempotency
    AD->>ESB: Canonical transfer command
    ESB->>T24: T24 API/OFS request
    T24->>GL: Create journal/posting
    T24-->>ESB: Response code
    ESB-->>AD: Mapped result
    AD-->>MB: Stable status
    MB-->>U: Success / failed / pending
```

Trạng thái giao dịch an toàn

```

stateDiagram-v2
    [*] --> INIT
    INIT --> VALIDATED
    VALIDATED --> PENDING_AUTH: maker-checker required
    VALIDATED --> POSTED: auto authorized
    PENDING_AUTH --> POSTED: approved
    PENDING_AUTH --> FAILED: rejected
    POSTED --> COMPLETED
    POSTED --> UNKNOWN: response timeout
    UNKNOWN --> COMPLETED: query status says posted
    UNKNOWN --> FAILED: query status says not posted
    COMPLETED --> REVERSED: business reversal
```

Vòng đời tiền gửi có kỳ hạn

```
stateDiagram-v2
    [*] --> Created
    Created --> Funded
    Funded --> Active
    Active --> Matured
    Active --> EarlyWithdrawal
    Matured --> Closed
    Matured --> RolledOver
    RolledOver --> Active
    EarlyWithdrawal --> Closed
```

COB/EOD

```
flowchart LR
    A[Cut-off / close online window] --> B[Pre-check]
    B --> C[Interest accrual]
    C --> D[Charges / fees]
    D --> E[Loan aging / overdue]
    E --> F[GL consolidation]
    F --> G[Reports / DWH]
    G --> H[Open next business date]
```

14. Nguồn tham khảo

Tài liệu này kết hợp kiến thức nền tảng banking domain, sơ đồ/ghi chú do bạn cung cấp, và các nguồn công khai/official dưới đây. Một số mô tả về T24/Temenos Transact được khái quát hóa ở mức kiến trúc, không thay thế tài liệu triển khai nội bộ của từng ngân hàng.

Mã	Nguồn	Link/ghi chú
[S1]	Temenos blog - "T24 is now Temenos Transact": Temenos công bố đổi tên core banking product từ Temenos T24 Transact sang Temenos Transact.	https://www.temenos.com/blog/t24-is-now-temenos-transact/
[S2]	Temenos Core Banking product page: mô tả modular core capabilities, cloud-native, end-to-end banking capabilities.	https://www.temenos.com/products/core-banking/
[S3]	Temenos Developer - Transact APIs: mô tả Arrangement Architecture cho accounts, deposits, loans và reusable business components.	https://developer.temenos.com/transact-apis
[S4]	Temenos Developer - Accounting Events Lifecycle Guide: mô tả business events và accounting events/journal entries.	https://developer.temenos.com/article/accounting-events-lifecycle-guide
[S5]	Temenos Developer - Account Events Lifecycle Guide: sample lifecycle cho savings/current accounts.	https://developer.temenos.com/article/account-events-lifecycle-guide
[S6]	Temenos Developer - Term Deposit Events Lifecycle Guide: creation, funding, early withdrawal, rollover.	https://developer.temenos.com/article/term-deposit-events-lifecycle-guide
[S7]	Temenos Developer - Retail Enterprise Services: create current, savings, notice accounts with limits, interest spreads, statements, joint holding.	https://developer.temenos.com/solutions/retail-enterprise-service
[S8]	Temenos Transact Data Hub/Operational Data Store: near real-time ODS, offload reporting/enquiry workloads.	https://developer.temenos.com/operational-data-store
[S9]	ISO 8583:2023: common interface and message format for financial-transaction-card-originated messages between acquirers and issuers.	https://www.iso.org/standard/79451.html
[S10]	ISO 20022: universal financial industry message scheme and common platform for financial messages.	https://www.iso20022.org/iso-20022
[S11]	PCI Security Standards Council - PCI DSS: security requirements for environments storing, processing or transmitting payment account data.	https://www.pcisecuritystandards.org/standards/
[S12]	OWASP API Security Top 10 2023: API risks such as Broken Object Level Authorization and Broken Authentication.	https://owasp.org/www-project-api-security/
[U1]	Core Banking Architecture note/chat and diagrams provided in this conversation.	Local project context

Mã	Nguồn	Link/ghi chú
[U2]	Accounting.md note provided in this conversation: journal, general ledger, balance sheet, P&L basics.	Local project context